MemSolve: From Sequential Memory Updates to Global Equilibrium

Zhaoyang Wang
zhaoy@uir.edu.cn

Working draft — October 21, 2026

Abstract

Gated Delta Networks (GDNs) have advanced causal sequence modeling by writing prediction errors back into memory to learn and refine associations. In bidirectional settings, the entire context is available to build a shared memory. However, LION's bidirectional DeltaNet recurrences and equivalent chunkwise form correct prefix and suffix memories separately. The direct full-attention form requires a sequence-sized matrix inverse, whose cubic cost undermines the efficiency of compact memory. We propose MemSolve, which directly solves a joint error-correction problem over all positions to build a compact equilibrium memory. Writing and reading this memory use the full context, coupling information on both sides of each token. We prove that single-layer bidirectional scans with local parameterization and linear merging cannot represent all such interactions. Our analysis also establishes a unique equilibrium and a contractive readout for each fixed write–read frame. Our hardware-aware algorithm supports exact sequence parallelism across devices and linear sequence complexity at fixed rank. At 8,192 tokens, its complete mixer is $7.5\times$ faster than FlashAttention and $7.9\times$ faster than BiMamba in single-GPU forward–backward execution. On ImageNet-1K, our Base model reaches 82.7% Top-1 with a plain non-hierarchical backbone and just 78M parameters. It surpasses ViG-B and ViT^3-B with approximately 13% fewer parameters. The same operator powers a BERT-Base-scale encoder with an average GLUE development score of 81.9.

1 Introduction

Gated Delta Networks (GDNs) have advanced causal sequence modeling by writing prediction errors back into memory to learn and refine associations [42, 23, 16]. Self-attention retains an explicit context for retrieval [35], while linear attention and state-space models (SSMs) compress it into a fixed-size memory [22, 13]. Stored associations can interfere within this compact state. The delta rule addresses this interference by predicting content from memory and writing back the prediction error [30, 43]. Each write can therefore revise the memory's existing predictions. GDNs enrich this correction with control over forgetting and writing.

Error correction can be viewed as training an internal predictor to learn associations from context. Test-time training (TTT) develops this view using trainable inner models [32]. Intention turns regression solves into composable neural modules [11]. MesaNet uses regression solves to construct causal memory states [36]. With the full context available, internal learning need not follow token order. In vision experiments, ViT^3 finds that full-batch updates using all tokens outperform sequential mini-batch updates [15]. Together, these works motivate our joint error-correction approach: all associations act on one memory state, whose equilibrium is solved directly rather than approached through a finite number of inner updates.

The challenge is to retain the efficiency of compact memory. LION's bidirectional DeltaNet recurrences correct prefix and suffix memories separately; an equivalent chunkwise computation preserves these updates [1]. The direct full-attention form discussed there requires a sequence-sized matrix inverse, whose cubic cost undermines that efficiency. A joint error-correction design must therefore address both how associations determine memory and how that memory is computed.

We propose **MemSolve**, which jointly designs full-context error correction and token readout. It directly solves for a compact equilibrium memory, then reads this memory back as a low-rank contextual correction to token content. Our main contributions are as follows.

Global Equilibrium (§3.1, §3.2). We jointly design the memory's error-correction equation and token readout. An input-dependent soft frame provides shared write–read coordinates. A matching regularizer reduces the memory equation to a learned compact core shared across inputs. An accretive parameterization guarantees a unique equilibrium. The same frame reads the solution back to tokens. The readout combines a content branch with a low-rank contextual correction. For each fixed frame, we prove that the token map is contractive and compute its exact singular values in compact space.

Beyond Independent Directional Scans (§3.3). MemSolve is not a reparameterization of a single bidirectional SSM mixer with local coefficients and linear directional merging. Such a mixer adds separate prefix and suffix responses. MemSolve instead normalizes its frame using the full context. Even with the center token fixed, a suffix change can alter how prefix content reaches that token.

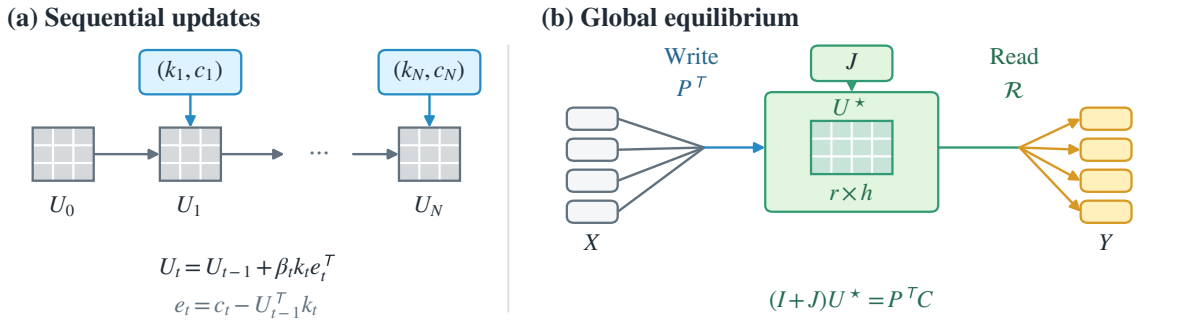


Figure 1: **From sequential memory updates to global equilibrium.** (a) Ordered updates correct one association at a time. (b) MemSolve combines all associations in one shared equilibrium and reads it back to every token. The core J is learned; the frame P and content C depend on the input. Section 3 derives the operator.

Theorem 3.3 proves this separation through a three-token counterexample.

Parallel Algorithm (§4). We develop a parallel algorithm that preserves the operator. It organizes the forward pass into compact statistic reductions, a compact solve, and independent token readouts. The forward pass exchanges only these statistics across sequence shards. The learned core is shared across inputs and evaluated once per head in each forward pass. Selective fusion avoids storing the token-sized frame and reduces GPU memory traffic. Computation scales linearly with sequence length at fixed rank. In our length sweep, MemSolve overtakes FlashAttention at 512 tokens for complete-mixer forward and backward passes. At 8,192 tokens, it is 7.5× faster than FlashAttention and 7.9× faster than BiMamba on one GPU (§5.3).

Architectures and Results (§5). We use the same operator in a plain non-hierarchical visual backbone and a BERT-style language encoder. On ImageNet-1K, Small reaches 81.0% Top-1 with 20M parameters, and Base reaches 82.7% with 78M. Base surpasses ViG-B and ViT^3-B with approximately 13% fewer parameters. Under their respective training recipes, Small also reaches the same validation accuracy in fewer epochs than the author-released DeiT-S run. For language, we match BERT-Base's parameter budget and obtain an average GLUE development score of 81.9.

2 Error-Correcting Memories

Error-correcting memories learn associations by revising their predictions. We first express this update as internal learning, then consider how to use all associations jointly when the full context is available.

2.1 Learning and refining associations

Let $U \in \mathbb{R}^{r\times h}$ be a memory that predicts content $c_i \in \mathbb{R}^h$ from a key $k_i \in \mathbb{R}^r$ as $U^\mathsf{T} k_i$. An additive write accumulates $k_i c_i^\mathsf{T}$. The basic delta rule instead computes a prediction error and writes it back into memory [30, 43]:

$$\begin{aligned} e_i &= c_i - U_{i-1}^\mathsf{T} k_i, \\ U_i &= U_{i-1} + \beta_i k_i e_i^\mathsf{T}. \end{aligned} \tag{1}$$

Here β_i controls the write strength. Unlike an additive write, this update depends on the memory's current prediction. It revises stored associations through their residual errors. GDNs add adaptive forgetting [42]; later variants introduce channel-wise forgetting and separate erase/write controls [23, 16].

Equation (1) is also a gradient step on

$$\ell_i(U) = \tfrac{1}{2}\left\|U^\mathsf{T} k_i - c_i\right\|_2^2, \tag{2}$$

whose gradient is $-k_i(c_i - U^\mathsf{T} k_i)^\mathsf{T}$. Each token thus supplies a training example for an internal predictor. The sequential update evaluates its error at the state reached so far.

2.2 From sequential updates to full-batch learning

TTT generalizes this interpretation to trainable inner models [32]. The sequence supplies training examples, and the hidden state holds the model learned from them. Test-time regression relates several memory updates through their regression objectives [38]. Internal learning can also use a regression solve. Intention fits embedded key–value pairs and reads the fitted state at query embeddings [11]. MesaNet constructs causal memory states by applying conjugate gradients to regression over the available context [36].

ViT^3 studies internal learning when the full input is visible [15]. Its full-batch update uses all token associations at the same inner-model state. Sequential mini-batches instead change that state between groups, so earlier groups affect the updates made by later groups. ViT^3 finds full-batch updates more effective in its vision experiments. Further inner passes can improve accuracy, but reduce throughput and can introduce training instability.

Full-batch updates and direct solves suggest a common direction: evaluate all association errors at one state and solve for their balance. Section 3 develops this joint error-correction problem together with its token readout.

2.3 Bidirectional access and token interactions

BiMamba, Hydra, and LION already let a token receive information from both sides [47, 20, 1]. Access to both sides does not determine how they interact. The question is whether changing a suffix token can alter how prefix content contributes to the output at a fixed center token.

For independent directional scans with local parameterization and linear merging, fixing the current token leaves an output of the form $y_i = f_i(x_{1:i}) + g_i(x_{i:N})$. Both sides contribute, but a suffix change cannot modify the prefix response. Section 3.3 uses this additive structure to establish a separation from MemSolve.

LION's Appendix B.7 gives independent bidirectional DeltaNet recurrences and discusses a chunkwise extension [1]. The directional memories perform error correction separately. An equivalent chunkwise implementation reorganizes the computation without changing these updates. The direct full-attention expression discussed there also involves a sequence-sized matrix inverse, with cubic cost for a dense evaluation. This motivates a design that lets full-context statistics determine token interactions while keeping the memory solve compact.

3 MemSolve

MemSolve jointly designs the memory equation and its token readout. An input-dependent frame supplies their shared coordinates. We match the regularizer to this frame so that the equilibrium uses a learned compact core shared across inputs. All equations describe one head with N valid tokens, rank r, and content width h. The theoretical results use exact arithmetic. Appendix A gives the full proofs.

3.1 Joint error correction and compact equilibrium

Let $C \in \mathbb{R}^{N\times h}$ contain the content to be stored, and let $P \in \mathbb{R}^{N\times r}$ assign memory coordinates to each position. A memory $U \in \mathbb{R}^{r\times h}$ predicts content as PU. For row p_i^T of P and row c_i^T of C, let $e_i^\mathsf{T} = c_i^\mathsf{T} - p_i^\mathsf{T}U$ be the prediction error. Summing the error-correction writes gives

$$\underbrace{\begin{bmatrix} p_1 & \cdots & p_N \end{bmatrix}}_{P^\mathsf{T}} \underbrace{\begin{bmatrix} e_1^\mathsf{T} \\ \vdots \\ e_N^\mathsf{T} \end{bmatrix}}_{C-PU} = p_1 e_1^\mathsf{T} + \cdots + p_N e_N^\mathsf{T}. \tag{3}$$

All prediction errors are evaluated against the same memory U.

To control the joint correction, we add a positive-definite regularizer Γ_X and a learned skew-symmetric coupling Ω:

$$\dot{U} = P^\mathsf{T}(C - PU) - \Gamma_X U - \Omega U. \tag{4}$$

The regularizer damps the memory. The skew term couples its coordinates. Its contribution to the change in squared memory norm is zero: $\langle U, \Omega U\rangle_F = 0$. MemSolve obtains the memory by solving for the equilibrium of this field. The next step is to construct P and Γ_X so that this solve is well posed and its core can be shared across inputs.

A learned projection produces relation features $B \in \mathbb{R}^{N\times r}$ and content C from the input X. Define

$$\begin{aligned} A &= B/\sqrt{N}, \quad R^\mathsf{T}R = \mathbf{I}_r + A^\mathsf{T}A, \\ P &= AR^{-1}, \qquad Z = P^\mathsf{T}C. \end{aligned} \tag{5}$$

Here R is the upper Cholesky factor with positive diagonal. Scaling by $\sqrt{N}$ makes $A^\mathsf{T}A$ an average relation second moment. The identity term keeps the factorization positive definite and ensures $\|P\|_2 < 1$. We call P a *soft frame*: it provides shared write–read coordinates with bounded gain. Appendix A.1 gives its singular values and geometry.

Each head learns a core shared across samples:

$$J = LL^\mathsf{T} + \Omega, \qquad LL^\mathsf{T} \succ 0, \qquad \Omega^\mathsf{T} = -\Omega. \tag{6}$$

Its symmetric part is positive definite, so J is strictly accretive. We use a lower triangular L with positive diagonal; the unconstrained parameterization is given in Appendix B.1. We couple the regularizer to the frame:

$$\Gamma_X = \mathbf{I}_r + LL^\mathsf{T} - P^\mathsf{T}P \succ 0. \tag{7}$$

Positivity follows from $P^\mathsf{T}P \prec \mathbf{I}_r$. This choice cancels the frame Gram term in Eq. (4):

$$\dot{U} = Z - (P^\mathsf{T}P + \Gamma_X + \Omega)U = Z - (\mathbf{I}_r + J)U. \tag{8}$$

The equilibrium and its reflected map are therefore

$$\begin{aligned} U^\star &= (\mathbf{I}_r + J)^{-1}Z, \\ F &= 2(\mathbf{I}_r + J)^{-1} - \mathbf{I}_r \\ &= (\mathbf{I}_r + J)^{-1}(\mathbf{I}_r - J). \end{aligned} \tag{9}$$

Here and below, inverse notation denotes a linear solve. This cancellation removes the input dependence from the system matrix, while retaining it in the memory coordinates and aggregated content. Each input supplies its own P, Z, and $U^\star$. The learned J and F are shared, so one core factorization serves the batch. The frame factorization remains input-dependent.

The accretive core guarantees a unique memory and bounds its response to perturbations.

Proposition 3.1 (Well-posed memory)**.** *Let* $\mu = 1 + \sigma_{\min}(L)^2 > 1$*. Then Eq.* (8) *has the unique equilibrium in Eq.* (9)*, and*

$$\left\|(\mathbf{I}_r + J)^{-1}\right\|_2 \le \mu^{-1}, \qquad \left\|(\mathbf{I}_r + J)^{-\mathsf{T}}\right\|_2 \le \mu^{-1}. \tag{10}$$

Every trajectory obeys

$$\|U(t) - U^\star\|_F \le e^{-\mu t} \|U(0) - U^\star\|_F .$$

The same bound controls right-hand-side perturbations in the forward and adjoint solves. It does not depend on the number of tokens.

The matched regularizer also gives a variational interpretation. When $\Omega = 0$, the equilibrium minimizes

$$\mathcal{E}_X(U) = \tfrac{1}{2} \|PU - C\|_F^2 + \tfrac{1}{2} \operatorname{tr}(U^\mathsf{T} \Gamma_X U). \tag{11}$$

With nonzero Ω, it solves a strongly monotone linear equation. The same existence guarantee holds, although the field is no longer the gradient of this scalar objective.

3.2 Token readout and contraction

The readout returns memory through the same frame used for writing. We use the reflected response $2U^\star - Z = FZ$ and combine it with a complementary content response:

$$Y = \eta(\mathbf{I}_N - PP^\mathsf{T})C + PFP^\mathsf{T}C \tag{12}$$

$$= \underbrace{\eta C}_{\text{content}} + \underbrace{P(F - \eta \mathbf{I}_r)P^\mathsf{T}C}_{\text{contextual correction}}. \tag{13}$$

Here $\eta = \tanh(\eta_{\text{raw}}) \in (-1, 1)$ is learned per head. Equivalently, $Y = \eta C + P[2U^\star - (1 + \eta)Z]$. The reflection produces a strict contraction F. The complementary term extends this gain bound to token space (Theorem 3.2). The resulting output is a content branch plus a contextual correction of rank at most r.

Theorem 3.2 (Contractive compact and token maps)**.** *The map* F *in Eq.* (9) *satisfies*

$$\mathbf{I}_r - F^\mathsf{T}F = 4(\mathbf{I}_r + J)^{-\mathsf{T}} LL^\mathsf{T} (\mathbf{I}_r + J)^{-1} \succ 0. \tag{14}$$

For a realized frame P*, define*

$$K_X = \eta(\mathbf{I}_N - PP^\mathsf{T}) + PFP^\mathsf{T}. \tag{15}$$

Then $\|K_X\|_2 \le \max\{|\eta|, \|F\|_2\} < 1$ *and* $\operatorname{rank}(K_X - \eta \mathbf{I}_N) \le r$*. Moreover,* $K_X v = \eta v$ *for every* $v \in \ker(P^\mathsf{T})$*.*

The token bound follows by compressing a block-diagonal contraction. Let $D = (\mathbf{I}_N - PP^\mathsf{T})^{1/2}$ and stack $V_0 = [D^\mathsf{T}, P]^\mathsf{T}$. Then

$$\begin{aligned} V_0^\mathsf{T} V_0 &= \mathbf{I}_N, \\ K_X &= V_0^\mathsf{T} \operatorname{diag}(\eta \mathbf{I}_N, F) V_0. \end{aligned} \tag{16}$$

Thus V_0 preserves norms, and the compact gain bound carries over to the full token map.

The bound applies to $C \mapsto K_X C$ with P fixed. It does not bound derivatives through $P(X)$ or the surrounding residual block. Within this map, the content path retains a scaled response outside the relation span. The contextual correction acts within that span.

The shared frame permits nonsymmetric interactions through F. Indeed, the accretive Cayley parameterization covers every real strict contraction in r dimensions (Appendix A.3). The token map retains its rank and shared-frame constraints. Its exact singular values can be recovered from a matrix of size at most r, together with the complementary content response (Appendix A.4).

The default EQUILIBRIUM mode learns F through J. Setting $F = 0$ gives COMPLEMENT, which omits the core parameters:

$$Y_{\text{com}} = \eta(\mathbf{I}_N - PP^\mathsf{T})C = \eta(\mathbf{I}_N + AA^\mathsf{T})^{-1}C. \tag{17}$$

The second equality is the Woodbury identity [12] applied to the complementary content map. Both expressions retain input-dependent context mixing. Both modes learn the frame, content, output projection, and η. Equilibrium starts from the Complement map and learns the additional compact interaction; Appendix B.1 specifies initialization. Head outputs are concatenated and passed through a learned output projection.

Optional rotary value encoding. Relative position can enter through the content pathway without separating write and read coordinates. Following RoVE [10], we rotate content before aggregation and apply the inverse rotation at readout. For column vectors c_i and rotations $O_i = O(t_i)$ at positions t_i,

$$y_i = \eta c_i + \sum_j D_{ij} O_i^\mathsf{T} O_j c_j, \qquad D = P(F - \eta \mathbf{I}_r)P^\mathsf{T}. \tag{18}$$

At a fixed frequency scale, $O_i^\mathsf{T} O_j = O(t_j - t_i)$ encodes relative offset. This extension adds no learned parameters and preserves the fixed-frame gain bound. Appendix B.7 gives the compact computation and settings.

3.3 Interactions beyond independent directional scans

The preceding analysis fixes the frame and studies the readout. We now examine the token interactions that arise as the frame changes with the input.

We compare MemSolve with a single bidirectional state-space mixer whose coefficients depend on the current token and whose directional outputs merge linearly:

$$\begin{aligned} h_i^+ &= A^+(x_i) h_{i-1}^+ + b^+(x_i), \\ h_i^- &= A^-(x_i) h_{i+1}^- + b^-(x_i), \\ y_i &= C^+(x_i) h_i^+ + C^-(x_i) h_i^- + d(x_i). \end{aligned} \tag{19}$$

Its boundary states are input-independent. The coefficients may be nonlinear functions of the current token,

and the state dimension is unrestricted. Once x_i is fixed, the output separates into $y_i = f_i(x_{1:i}) + g_i(x_{i:N})$. Changing the suffix cannot change the prefix response. MemSolve's global frame normalization can change that response, yielding the following separation.

Theorem 3.3 (Separation from locally parameterized bidirectional SSMs)**.** *For every positive rank r, the MemSolve family contains a map that cannot be represented by Eq.* (19) *for all inputs, irrespective of that mixer's state dimension. The result holds for both Equilibrium and Complement.*

The proof uses three tokens. Set $A = (1, 1, t)^\mathsf{T}$, $C = (s, 0, 0)^\mathsf{T}$, $F = 0$, and $\eta \neq 0$. Equation (17) gives

$$y_2 = -\frac{\eta s}{3 + t^2}. \tag{20}$$

The first two tokens stay fixed as t varies. Changing only the third token changes the coefficient $-\eta/(3 + t^2)$ that carries the first token's content to the second.

For the four choices $(s, t) \in \{0, 1\}^2$, the mixed finite difference $y_2(1, 1) - y_2(1, 0) - y_2(0, 1) + y_2(0, 0)$ is $\eta/12$ for Eq. (20) and zero for every additive prefix–suffix response. Appendix A.5 gives the full proof and extends the example to any rank. The theorem concerns individual mixers with local coefficients and linear directional merging. Stacks, nonlinear merges, and globally conditioned coefficients define different model classes.

4 Hardware-Aware Parallel Computation

The mathematical construction uses a token-sized frame P. Our algorithm avoids storing this matrix. It reduces the input to two compact statistics, solves in memory space, and reads out token rows in parallel. Both operator modes follow the same computation.

4.1 Reduce, solve, and read out

Substituting $P = AR^{-1}$ into Eq. (13) gives

$$\begin{aligned} G = A^\mathsf{T}A, \qquad H = A^\mathsf{T}C, \\ R^\mathsf{T}R = \mathbf{I}_r + G, \qquad Z = R^{-\mathsf{T}}H, \\ W = R^{-1}(F - \eta\mathbf{I}_r)Z, \\ Y = \eta C + AW. \end{aligned} \tag{21}$$

The Gram statistic G determines the frame normalization, while H aggregates relation–content associations. Together they determine W. Once W is available, each output row uses only its local rows of A and C. Algorithm 1 gives this computation. Equilibrium and Complement use the same steps; Complement takes $F = 0$.

Algorithm 1 MemSolve: compact forward computation (one sample and head)

Input: Relations $B \in \mathbb{R}^{N\times r}$, content $C \in \mathbb{R}^{N\times h}$; shared F, scalar η; $N > r$ valid tokens
Output: Token output $Y \in \mathbb{R}^{N\times h}$

$A \leftarrow B/\sqrt{N}$
$G \leftarrow A^\mathsf{T}A$; $H \leftarrow A^\mathsf{T}C$ ▷ Parallel token reductions
$R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$ ▷ $R^\mathsf{T}R = \mathbf{I}_r + G$
$Z \leftarrow \mathrm{solve}(R^\mathsf{T}, H)$
$W \leftarrow \mathrm{solve}(R, (F - \eta\mathbf{I}_r)Z)$
return $\eta C + AW$ ▷ Parallel token readout

The learned F is evaluated once per head in each forward pass. Each sample then requires $O(Nr^2 + Nrh + r^3 + r^2h)$ arithmetic and $O(r^2 + rh)$ compact storage per head. Token storage is linear in N. Input and output projections also have linear sequence cost at fixed width. For physical length $N \leq r$, an equivalent token-space solve avoids subtraction in the complementary term. Its dimension is at most r. Appendix B.2 gives both length regimes and padding handling.

4.2 Exact sequence parallelism

The two statistics are sums over token rows. Partition those rows across p devices and normalize each shard as $A_s = B_s/\sqrt{N}$, using the same global valid count N. For two shards, expanding $A^\mathsf{T}[\,A \;\; C\,]$ shows which contributions combine:

$$\begin{aligned} [\,G \;\; H\,] &= \begin{bmatrix} A_1^\mathsf{T} & A_2^\mathsf{T} \end{bmatrix} \begin{bmatrix} A_1 & C_1 \\ A_2 & C_2 \end{bmatrix} \\ &= A_1^\mathsf{T}[\,A_1 \;\; C_1\,] + A_2^\mathsf{T}[\,A_2 \;\; C_2\,] \\ &= [\,G_1 + G_2 \quad H_1 + H_2\,]. \end{aligned} \tag{22}$$

Here $G_s = A_s^\mathsf{T}A_s$ and $H_s = A_s^\mathsf{T}C_s$. For p shards, a sum reduction gives each device $G = \sum_s G_s$ and $H = \sum_s H_s$. Each device computes the same W and reads out its own rows as $Y_s = \eta C_s + A_s W$. The statistic payload is $O(r^2 + rh)$ per sample and head, independent of sequence length. No token activation gather is needed.

The backward pass likewise reduces a compact readout adjoint before computing local token gradients. Outputs and reduced gradients equal the unsharded computation in real arithmetic; floating-point reduction order can change rounding. Algorithm 5 specifies the reductions and the treatment of shared parameter gradients.

4.3 Selective fusion

We selectively fuse token operations around the compact statistics and readout. Compact factorizations remain separate so that specialized solvers can handle them. The backward pass reuses the compact factors and reduces core gradients across the batch without storing one core adjoint per sample.

The GPU implementation uses Triton [33] for tiled token operations, cuSOLVERDx for compact factorizations, and cuBLASLt for projection gradients. Compact solves and adjoints retain FP32 storage. The readout splits sensitive coefficients into a BF16 leading word and a BF16 residual word [17]. Both products accumulate in FP32 before output rounding. This improves coefficient precision while retaining Tensor Core products. Appendix B.6 specifies precision, scheduling, and the surrounding compilation and CUDA Graph execution.

5 Experiments

We evaluate Equilibrium in a plain visual backbone and a BERT-style encoder. The visual experiments compare accuracy, parameter count, and convergence. The language experiments assess transfer after masked-language pretraining. RoVE is disabled in both settings.

5.1 ImageNet classification

Model and protocol. MemSolve-ViT follows the non-hierarchical ViT design [9], with non-overlapping 16×16 patches and twelve constant-width blocks. Each block adds residual 3×3 depthwise CPE [4], then applies pre-LayerNorm MemSolve and GELU FFN branches with residual connections and stochastic depth [19] (Fig. 2). A final token-wise LayerNorm precedes mean pooling and a linear classifier.

We train from scratch on ImageNet-1K [29] for 300 epochs at 224×224 with a recipe derived from plain ViT^3. Both models use AdamW [26] and cross-entropy, with global batch 2,048 and peak learning rate 2×10^{-3}. We evaluate EMA weights with decay 0.9999 and report the checkpoint with the highest validation Top-1. Appendix C.1 gives the architecture and training recipe.

Higher accuracy in smaller models. With a plain, twelve-layer non-hierarchical backbone, MemSolve-ViT-B reaches 82.7% Top-1 with just 78M parameters, surpassing ViG-B and ViT^3-B, which use 89M and 90M parameters (Table 1). It also exceeds DeiT-B and Vim-B by 0.9 and 0.8 percentage points, respectively, while using fewer parameters than either model. These results establish compact equilibrium memory as an effective token mixer for visual recognition with a simple backbone.

The advantage extends to Small scale. MemSolve-ViT-S reaches 81.0% with 20M parameters, the smallest parameter count among the Small models in Table 1. It exceeds DeiT-S, Vim-S, and LION-d$^\natural$-S by 1.2, 0.7, and 1.1 percentage points, respectively. The gains over these baselines therefore hold at both model scales.

Earlier gains in validation accuracy. MemSolve's advantage emerges well before the end of training. MemSolve-ViT-S reaches 70% Top-1 at epoch 65, whereas the author-released DeiT-S run reaches it at epoch 118 [34]. This reduces the required epoch count by approximately 45% (Fig. 3). At 75% Top-1, MemSolve reduces the required epochs from 192 to 148, a reduction of approximately 23%. MemSolve thus reaches the same accuracy targets in fewer epochs while attaining higher final accuracy. This convergence advantage suggests potential benefits for large-scale training, where repeated passes over the data are costly.

Method	Type	#Params	Top-1
ConvNeXt-S (iso.) [25]	ConvNet	22M	79.7
DeiT-S [34]	Transformer	22M	79.8
Vim-S [47]	SSM	26M	80.3
Vim-S† [47]	SSM	26M	81.4
LION-d$^\natural$-S [1]	Linear	22M	79.9
ViG-S [24]	Linear	23M	81.7
ViT^3-S [15]	TTT	24M	81.6
MemSolve-ViT-S	Equilibrium	20M	81.0
ConvNeXt-B (iso.) [25]	ConvNet	87M	82.0
ViT-B [20]	Transformer	87M	80.6
DeiT-B [34]	Transformer	86M	81.8
Vim-B [47]	SSM	98M	81.9
Vim-B† [47]	SSM	98M	83.2
Hydra-ViT-B [20]	SSM	91M	81.6
LION-d$^\natural$-B [1]	Linear	86M	80.2
ViG-B [24]	Linear	89M	82.6
ViT^3-B [15]	TTT	90M	82.6
MemSolve-ViT-B	Equilibrium	78M	82.7

Table 1: **Non-hierarchical models on ImageNet-1K.** Top-1 is in percent. All inputs are 224×224. †: additional long-sequence fine-tuning; ♮: multiple scans. Appendix C.1 details the reporting conventions.

5.2 Masked language modeling and GLUE

A parameter-matched encoder. We replace BERT's attention mixer with MemSolve in a 14-layer encoder with SwiGLU FFNs [31]. It has 112M parameters including the tied MLM head, close to the 110M BERT-Base scale [8]. The model uses BERT's post-LayerNorm residual structure and learned absolute positions. A centered residual depthwise convolution of width three precedes the relation/content projection in every layer. Appendix C.2 lists the model and pretraining configuration.

Pretraining. We follow the LION/M2-derived C4 masked-language setup [28, 1]: sequence length 128, global batch 4,096, 70,000 optimizer steps, and training mask probability 0.30. On a fixed 10,000-document validation subset with 15% masking, the selected checkpoint reaches 66.4% masked-token accuracy and masked-token NLL 1.6211 over 169,544 targets. The complete run takes 31.80 hours on two A800-80GB GPUs, including scheduled validation and checkpointing.

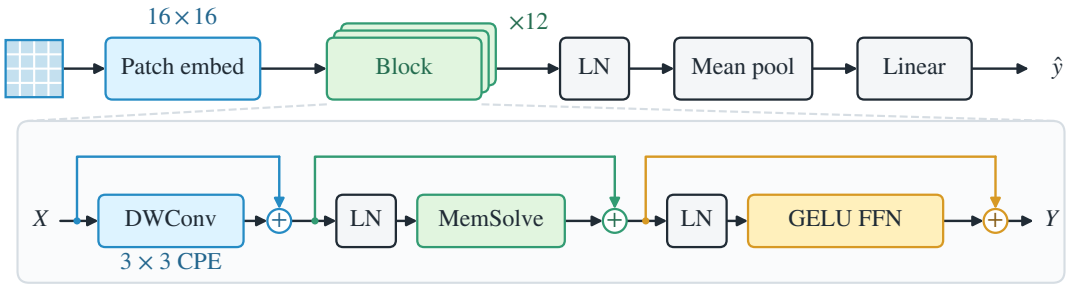


Figure 2: **MemSolve-ViT architecture and block structure.** CPE denotes conditional positional encoding.

Table 2: **GLUE development results at Base scale.** Avg. is the unweighted mean of eight task scores. Appendix C.3 gives metric definitions and evaluation details.

Model	MNLI	RTE	QQP	QNLI	SST-2	STS-B	MRPC	CoLA	Avg.
BERT	84.6	64.3	90.0	89.8	92.5	86.7	89.6	60.4	82.3
LION-LIT	82.5	63.5	89.7	89.3	91.7	87.2	89.4	49.2	80.3
LION-D	82.9	52.5	89.6	88.4	91.9	86.0	83.9	53.6	78.6
LION-S	83.2	53.5	89.4	88.9	93.0	37.7	77.9	53.2	72.1
MemSolve	83.0	74.2	89.8	88.6	91.6	89.2	91.0	47.5	81.9

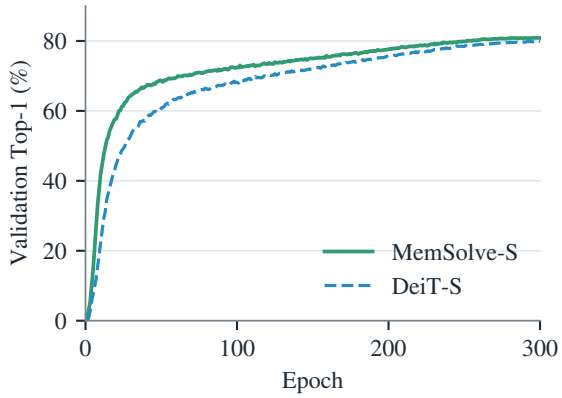


Figure 3: **ImageNet-1K convergence of MemSolve-S and DeiT-S.** Both curves evaluate non-EMA weights.

Downstream protocol. We fine-tune on eight GLUE tasks using the LION paper's M2-BASE task hyperparameters [37, 1]. MNLI, QQP, QNLI, SST-2, and CoLA start from the MLM checkpoint; RTE, MRPC, and STS-B start from the fine-tuned MNLI encoder. All tasks use length 128 and final-checkpoint development evaluation.

Closing the gap to BERT. The same operator also delivers competitive language understanding. At the BERT-Base scale, MemSolve reaches an average GLUE score of 81.9, exceeding all three LION M2-BASE variants (Table 2). Compared with the strongest LION variant, LION-LIT, it reduces the gap to BERT from 2.0 points to just 0.4 points. MemSolve also achieves the highest scores among the listed models on RTE, STS-B, and MRPC, exceeding BERT by 9.9, 2.5, and 1.4 points. CoLA remains its largest shortfall, at 47.5 versus BERT's 60.4. Together with the ImageNet results, these findings demonstrate that compact equilibrium memory supports strong representations in both vision and language.

5.3 Computational efficiency

We compare MemSolve with FlashAttention [6] and Vim's official BiMamba mixer [47] on one A800-80GB (Table 3). We time complete mixers, including input and output projections, using BF16 autocast and CUDA Graphs for all three implementations.

Speed. MemSolve's advantage grows with sequence length. In the fixed-width sweep, its forward–backward pass overtakes FlashAttention at 512 tokens. The speedup reaches 2.3× at 2,048 tokens and 28.2× at 32,768 tokens. At the latter length, it also outperforms the fused BiMamba implementation by 9.0×. The benefit extends to the 196-token visual shapes: MemSolve is 1.1–1.7× faster than FlashAttention and 8.4–12.8× faster than BiMamba across Tiny, Small, and Base. FlashAttention leads at 128 and 196 tokens in the small-batch sweep. It also has a faster forward-only pass for the Small and Base visual shapes; Appendix C.4 reports these measurements.

Memory. MemSolve has the lowest peak allocated memory in every measured setting. At 32,768 tokens, its forward–backward pass uses 2.58 GiB, compared with 2.87 GiB for FlashAttention and 6.00 GiB for BiMamba. This reduces memory by 10% and 57%, respectively. The compact solve thus supports long sequences with both lower latency and lower memory use.

Table 3: **Mixer forward–backward latency and peak memory.** All three use CUDA Graphs on one A800-80GB. Lower is better.

	MemSolve		FlashAttention		BiMamba	
Setting	ms	MiB	ms	MiB	ms	MiB
Visual shapes: 196 tokens						
Tiny	**0.620**	**169.2**	1.066	193.9	7.921	359.6
Small	**1.248**	**308.1**	1.783	321.1	15.565	664.7
Base	**1.900**	**329.2**	2.087	332.1	16.052	680.3
Length sweep: batch 4, width 768						
128 tokens	0.374	**64.4**	**0.267**	76.2	0.987	103.2
196 tokens	0.400	**66.2**	**0.332**	78.4	1.363	115.5
512 tokens	**0.524**	**90.8**	0.564	120.4	1.775	172.6
2,048 tokens	**0.997**	**213.3**	2.295	236.4	5.790	455.7
8,192 tokens	**2.792**	**698.2**	20.907	775.6	21.948	1592.8
32,768 tokens	**10.047**	**2642.2**	283.266	2940.1	90.289	6140.7

6 Discussion and Limitations

MemSolve concentrates contextual changes in a learned relation span. Rank controls the number of modified directions and the cost of the compact systems. Outside that span, the operator retains a scalar content response. Sharing write and read coordinates supports the contraction guarantee, but it also ties where information is stored to where it is retrieved. These choices define the operator's capacity and computational tradeoff.

The experiments show that this construction learns useful representations in vision and language. The language experiments train and fine-tune at 128 tokens with learned absolute positions; task accuracy beyond that length remains to be measured.

7 Conclusion

MemSolve turns joint error correction into a compact equilibrium solve. Its frame and matched regularizer yield a shared core, while each input determines the memory and its token readout. The readout combines a content branch with a low-rank contextual correction and is contractive for each fixed frame. Global frame normalization allows a suffix token to change how prefix content reaches a fixed position. This interaction separates MemSolve from single bidirectional scans with local coefficients and linear merging. Two compact statistics support linear sequence cost and exact sequence parallelism at fixed rank. Results in vision and language demonstrate the effectiveness of this shared operator.

References

[1] Arshia Afzal, Elias Abad Rocamora, Leyla Naz Candogan, Pol Puigdemont, Francesco Tonin, Yongtao Wu, Mahsa Shoaran, and Volkan Cevher. Linear attention for efficient bidirectional sequence modeling. In *Advances in Neural Information Processing Systems*, volume 38, pages 78994–79039, 2025.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. CMS Books in Mathematics. Springer, Cham, second edition, 2017.

[4] Xiangxiang Chu, Zhi Tian, Bo Zhang, Xinlong Wang, and Chunhua Shen. Conditional positional encodings for vision transformers. In *International Conference on Learning Representations*, 2023.

[5] Ekin D. Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *Advances in Neural Information Processing Systems*, volume 33, pages 18613–18624, 2020.

[6] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *Proceedings of the International Conference on Learning Representations*, 2024.

[7] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 2024.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.

[9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16×16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[10] Alejandro García-Castellanos, Maurice Weiler, and Erik J. Bekkers. RoVE: Rotary value embeddings attention for relative position-dependent value pathways. *arXiv preprint arXiv:2606.11275*, 2026.

[11] Marta Garnelo and Wojciech Marian Czarnecki. Exploring the space of key-value-query models with intention. *arXiv preprint arXiv:2305.10203*, 2023.

[12] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, fourth edition, 2013.

[13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024.

[14] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[15] Dongchen Han, Yining Li, Tianyu Li, Zixuan Cao, Ziming Wang, Jun Song, Yu Cheng, Bo Zheng, and Gao Huang. ViT^3: Unlocking test-time training in vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 51–61, 2026.

[16] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated DeltaNet-2: Decoupling erase and write in linear attention. *arXiv preprint arXiv:2605.22791*, 2026.

[17] Greg Henry, Ping Tak Peter Tang, and Alexander Heinecke. Leveraging the bfloat16 artificial intelligence datatype for higher-precision computations. In *Proceedings of the 26th IEEE Symposium on Computer Arithmetic*, pages 69–76. IEEE, 2019.

[18] Elad Hoffer, Tal Ben-Nun, Itay Hubara, Niv Giladi, Torsten Hoefler, and Daniel Soudry. Augment your batch: Improving generalization through instance repetition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8129–8138, 2020.

[19] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *Computer Vision – ECCV 2016*, pages 646–661. Springer International Publishing, 2016.

[20] Sukjun Hwang, Aakash Lahoti, Ratish Puduppully, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. In *Advances in Neural Information Processing Systems*, volume 37, pages 110876–110908, 2024.

[21] Timothy Kassis, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. *arXiv preprint arXiv:2609.00065*, 2026.

[22] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.

[23] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.

[24] Bencheng Liao, Xinggang Wang, Lianghui Zhu, Qian Zhang, and Chang Huang. ViG: Linear-complexity visual sequence learning with gated linear attention. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(5):5182–5190, 2025.

[25] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022.

[26] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[27] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.

[28] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[29] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

[30] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 2021.

[31] Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[32] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522. PMLR, 2025.

[33] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19. ACM, 2019.

[34] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers and distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 2021.

[35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[36] Johannes von Oswald, Nino Scherrer, Seijin Kobayashi, Luca Versari, Songlin Yang, Maximilian Schlegel, Kaitlin Maile, Yanick Schimpf, Oliver Sieberling, Alexander Meulemans, Rif A. Saurous, Guillaume Lajoie, Charlotte Frenkel, Razvan Pascanu, Blaise Agüera y Arcas, and João Sacramento. MesaNet: Sequence modeling by locally optimal test-time training. In *International Conference on Learning Representations*, pages 26397–26447, 2026.

[37] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.

[38] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: A unifying framework for designing sequence models with associative memory. *Journal of Machine Learning Research*, 27(169):1–41, 2026.

[39] Ezra Winston and J. Zico Kolter. Monotone operator equilibrium networks. In *Advances in Neural Information Processing Systems*, volume 33, pages 10718–10728, 2020.

[40] Haocheng Xi, Yiming Xie, Hexu Zhao, Yiwen Zhang, Michael Liu, Thomas Creavin, Kurt Keutzer, Xiuyu Li, Zhaoyang Lv, Chenfeng Xu, and Haiwen Feng. Video DeltaNet: A video-native hybrid attention for livestream video generation. *arXiv preprint arXiv:2609.20744*, 2026.

[41] Zipeng Xiao, Zhongkai Hao, Bokai Lin, Zhijie Deng, and Hang Su. Improved operator learning by orthogonal attention. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 54288–54299. PMLR, 2024.

[42] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, pages 29687–29707, 2025.

[43] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, volume 37, pages 115491–115522, 2024.

[44] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. CutMix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6023–6032, 2019.

[45] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018.

[46] Zhun Zhong, Liang Zheng, Guoliang Kang, Shaozi Li, and Yi Yang. Random erasing data augmentation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(7):13001–13008, 2020.

[47] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62429–62442. PMLR, 2024.

Appendix

The appendix provides supporting theory, implementation details, and experimental protocols. It is organized as follows:

- Appendix A presents the proofs and structural properties of MemSolve.
- Appendix B gives the algorithms, numerical details, and optional RoVE extension.
- Appendix C describes the vision and language protocols and results.
- Appendix D expands the discussion of related work.

A Proofs and Structural Details

We first establish the frame geometry. It gives the positive regularizer used in the memory equation. We then prove existence, contraction, and the compact spectrum, followed by the bidirectional-scan separation.

A.1 Soft-frame geometry

Since $R^\mathsf{T}R = \mathbf{I} + A^\mathsf{T}A$,

$$PP^\mathsf{T} = A(\mathbf{I} + A^\mathsf{T}A)^{-1}A^\mathsf{T}. \tag{23}$$

An SVD of A gives

$$\sigma_i(P) = \frac{\sigma_i(A)}{\sqrt{1+\sigma_i(A)^2}} < 1. \tag{24}$$

In particular, $P^\mathsf{T}P \prec \mathbf{I}_r$ and $PP^\mathsf{T} \prec \mathbf{I}_N$, including rank-deficient cases. The Woodbury identity yields

$$\mathbf{I}_N - PP^\mathsf{T} = (\mathbf{I}_N + AA^\mathsf{T})^{-1} \succ 0, \tag{25}$$

which proves Eq. (17) and positivity of Γ_X. The reduced QR factorization of $[A^\mathsf{T}, \mathbf{I}_r]^\mathsf{T}$ with a positive diagonal in its triangular factor has top block P. Thus the QR reference and the compact Cholesky construction agree in exact arithmetic.

The $\sqrt{N}$ normalization controls the scale of the relation statistic. Write b_i^T for row i of B. If $N^{-1}\sum_i \|b_i\|_2^2 \le c$, then $\|A\|_2^2 \le \|A\|_F^2 \le c$ and $\kappa_2(\mathbf{I} + A^\mathsf{T}A) \le 1 + c$, independently of N. Under this feature-energy bound, the condition number does not grow with sequence length. Accuracy at unseen lengths is a separate empirical question.

Without a positional module, the operator is permutation equivariant: permuting the rows of A, C leaves G, H unchanged and permutes the rows of Y. The visual and language backbones therefore supply position information outside the core operator.

A.2 Memory existence, sensitivity, and convergence

Write $M = \mathbf{I}_r + J$. For every real v,

$$v^\mathsf{T}Mv = v^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})v \ge \mu \|v\|_2^2, \qquad \mu = 1 + \sigma_{\min}(L)^2. \tag{26}$$

Cauchy–Schwarz gives $\|Mv\|_2 \ge \mu \|v\|_2$, so M is invertible and $\left\|M^{-1}\right\|_2 \le 1/\mu$. The same argument applies to M^T. For $E(t) = U(t) - U^\star$, $\dot{E} = -ME$ and

$$\frac{d}{dt}\|E\|_F^2 = -2\,\mathrm{tr}(E^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})E) \le -2\mu \|E\|_F^2. \tag{27}$$

Integration proves Proposition 3.1. For $\Omega = 0$, differentiating Eq. (11) gives $\nabla_U \mathcal{E}_X = MU - Z$ and Hessian $M \succ 0$ for each content column. When $\Omega \neq 0$, the nonsymmetric field need not be a scalar gradient, although the same convergence argument holds.

A.3 Cayley contraction and coverage

As $M = \mathbf{I} + J$ commutes with $\mathbf{I} - J$,

$$\mathbf{I} - F^{\mathsf{T}}F = M^{-\mathsf{T}}\left[M^{\mathsf{T}}M - (\mathbf{I}-J)^{\mathsf{T}}(\mathbf{I}-J)\right]M^{-1} \tag{28}$$

$$= 2M^{-\mathsf{T}}(J + J^{\mathsf{T}})M^{-1} = 4M^{-\mathsf{T}}LL^{\mathsf{T}}M^{-1} \succ 0. \tag{29}$$

Conversely, let F_0 be any real matrix with $\|F_0\|_2 < 1$. Then $\mathbf{I} + F_0$ is invertible and

$$J_0 = (\mathbf{I} - F_0)(\mathbf{I} + F_0)^{-1}, \qquad \mathrm{sym}(J_0) = (\mathbf{I} + F_0)^{-\mathsf{T}}(\mathbf{I} - F_0^{\mathsf{T}}F_0)(\mathbf{I} + F_0)^{-1} \succ 0. \tag{30}$$

Factor $\mathrm{sym}(J_0) = LL^{\mathsf{T}}$ and set $\Omega = (J_0 - J_0^{\mathsf{T}})/2$. The triangular/softplus parameterization can represent every such L and Ω. This is the classical accretive–Cayley correspondence [3]. It establishes coverage of all real strict contractions in compact space; the token map retains its shared-frame and rank constraints.

A.4 Token contraction and the exact compact spectrum

Equation (16) expresses K_X as an isometric compression of $\mathrm{diag}(\eta\mathbf{I}_N, F)$. Submultiplicativity of the spectral norm gives the bound in Theorem 3.2. The rank and nullspace statements follow directly from $K_X - \eta\mathbf{I}_N = P(F - \eta\mathbf{I}_r)P^{\mathsf{T}}$.

Proposition A.1 (Exact analysis in compact space)**.** *Let $P = QSV^{\mathsf{T}}$ be a thin singular value decomposition with $k = \mathrm{rank}(P)$, $Q^{\mathsf{T}}Q = V^{\mathsf{T}}V = \mathbf{I}_k$, and positive diagonal S. The token map is orthogonally similar to*

$$\left[\eta\mathbf{I}_k + SV^{\mathsf{T}}(F - \eta\mathbf{I}_r)VS\right] \oplus \eta\mathbf{I}_{N-k}. \tag{31}$$

Its singular values, including its exact spectral norm, can therefore be obtained from a matrix of size at most r, together with $N - k$ copies of $|\eta|$.

To prove the proposition, extend Q to an orthogonal basis $[Q, Q_\perp]$. In this basis the low-rank correction acts only on the first k coordinates, giving Eq. (31). If $k = 0$, the result is simply $\eta\mathbf{I}_N$; if $k = N$, the complementary block is absent. One may compute V and S from the positive eigenspace of $P^{\mathsf{T}}P$, so no $N \times N$ matrix or token-space SVD is required.

A.5 Proof of the bidirectional SSM separation

Fix the center token x_i in Eq. (19). Recursion with input-independent boundary states gives $y_i = f_i(x_{1:i}) + g_i(x_{i:N})$. For any prefix perturbation s and suffix perturbation t that leave x_i fixed, its mixed finite difference is

$$\Delta_{s,t}y_i = y_i(s_1, t_1) - y_i(s_1, t_0) - y_i(s_0, t_1) + y_i(s_0, t_0) = 0. \tag{32}$$

This conclusion holds for arbitrary coefficient functions and state size.

For MemSolve, take three tokens whose two coordinates independently control relation and content: $x_1 = (1, s)$, $x_2 = (1, 0)$, and $x_3 = (t, 0)$. A fixed relation projection with scale $\sqrt{3}$ gives $A = (1, 1, t)^{\mathsf{T}}$ after the normalization in Eq. (5); a content projection gives $C = (s, 0, 0)^{\mathsf{T}}$. With $F = 0$,

$$PP^{\mathsf{T}} = \frac{AA^{\mathsf{T}}}{1 + A^{\mathsf{T}}A}, \qquad y_2 = -\frac{\eta s}{3 + t^2}. \tag{33}$$

At $(s_0, s_1) = (0, 1)$ and $(t_0, t_1) = (0, 1)$,

$$\Delta_{s,t}y_2 = -\frac{\eta}{4} + \frac{\eta}{3} = \frac{\eta}{12} \neq 0. \tag{34}$$

Therefore no mixer of Eq. (19) represents this MemSolve map for all inputs. For any larger rank, append zero relation columns; the first Cholesky coordinate and the output above remain unchanged. Complement sets $F = 0$ by definition, and Equilibrium realizes $F = 0$ at $J = \mathbf{I}_r$. This proves Theorem 3.3.

The distinction concerns how inputs determine the map, not the rank of a matrix after fixing the input. A realized MemSolve matrix is low rank plus a diagonal and can admit other algebraic factorizations. Such a factorization does not supply the local coefficient dependence required by Eq. (19).

B Algorithms and Numerical Details

This appendix gives the core parameterization, the masked forward pass, and the analytic backward. It then specifies sequence-parallel reductions and floating-point execution. The pseudocode describes exact arithmetic; Section B.6 explains how the GPU kernels evaluate it.

B.1 Core parameters and initialization

Each head stores an unconstrained matrix $\Theta \in \mathbb{R}^{r\times r}$. Its lower and upper entries define the symmetric and skew parts of J:

$$L = \mathrm{tril}(\Theta, -1) + \mathrm{diag}(\mathrm{softplus}(\mathrm{diag}\,\Theta + \log(e-1))), \tag{35}$$

$$\Omega = \mathrm{triu}(\Theta, 1) - \mathrm{triu}(\Theta, 1)^{\mathsf{T}}, \qquad J = LL^{\mathsf{T}} + \Omega. \tag{36}$$

The diagonal of L is positive, so $LL^{\mathsf{T}} \succ 0$. At $\Theta = 0$, $L = \mathbf{I}_r$, $J = \mathbf{I}_r$, and $F = 0$. This initialization preserves the core's training signal: $\delta F = -\frac{1}{2}\delta J$ at $J = \mathbf{I}_r$. The additional response $PFP^{\mathsf{T}}C$ starts at zero, while the Complement map already mixes context. The core remains trainable at this initialization.

The per-head scalar is parameterized by $\eta = \tanh(\eta_{\mathrm{raw}})$ and initialized to 0.9. Floating-point execution uses

$$\eta_{\mathrm{fp}} = (1 - \epsilon_{\mathrm{mach}})\tanh(\eta_{\mathrm{raw}}), \qquad \frac{d\eta_{\mathrm{fp}}}{d\eta_{\mathrm{raw}}} = (1-\epsilon_{\mathrm{mach}})\frac{4q}{(1+q)^2}, \quad q = e^{-2|\eta_{\mathrm{raw}}|}. \tag{37}$$

Here ϵ_{mach} is the machine epsilon of the scalar computation dtype, which is FP32 in the CUDA path. The scale keeps the value inside $(-1, 1)$. Native $\tanh$ preserves small values near zero; the exponential derivative retains representable tail gradients after $\tanh$ rounds to an endpoint.

B.2 Forward computation

Algorithm 2 expands Algorithm 1 to masked batches and short sequences. The input and output projections are ordinary affine layers. Head outputs are concatenated, projected, and masked again so that output bias cannot populate padding.

Physical length is N; sample b has n_b valid tokens. Masks multiply rows and are fixed under differentiation. An all-masked sample uses a divisor of one and produces zero. We use $\mathrm{solve}(M, V)$ for the solution of $MU = V$ and $\mathrm{chol}_u(S)$ for the upper factor R satisfying $R^{\mathsf{T}}R = S$. Thus

$$\mathrm{cholsolve}(R, V) = \mathrm{solve}\big(R, \mathrm{solve}(R^{\mathsf{T}}, V)\big). \tag{38}$$

A general core system uses LU factorization. For $N \le r$, the complementary term uses the equivalent solve $\eta\,\mathrm{solve}(\mathbf{I}_N + AA^{\mathsf{T}}, C)$. This avoids subtracting nearly equal terms in $\eta(C - PP^{\mathsf{T}}C)$. Equilibrium adds its learned correction to that result. The largest factorization has dimension at most r in either regime.

Algorithm 2 MemSolve forward without materializing P (one head)

Input: $\{B_b \in \mathbb{R}^{N\times r}, C_b \in \mathbb{R}^{N\times h}, m_b \in \{0,1\}^N\}_{b=1}^s$; mode, η; $\Theta \in \mathbb{R}^{r\times r}$ for Equilibrium
Output: Head outputs $\{Y_b \in \mathbb{R}^{N\times h}\}_{b=1}^s$

if mode is Equilibrium **then**
  Construct L, Ω, J from Θ using Eq. (36)
  $M \leftarrow \mathbf{I}_r + J$; factor M once
  $F \leftarrow \mathrm{solve}(M, \mathbf{I}_r - J)$ ▷ Shared across this batch
else
  $F \leftarrow 0$ ▷ Complement
end if
for each sample b in parallel **do**
  $n_b \leftarrow \sum_i m_{b,i}$; $A \leftarrow (m_b \odot B_b)/\sqrt{\max(n_b, 1)}$; $C \leftarrow m_b \odot C_b$
  if $N \le r$ **then** ▷ Physical length determines the branch
    $R_N \leftarrow \mathrm{chol}_u(\mathbf{I}_N + AA^{\mathsf{T}})$; $Y_b \leftarrow \eta\ \mathrm{cholsolve}(R_N, C)$
    if mode is Equilibrium **then**
      $R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + A^{\mathsf{T}}A)$; $Z \leftarrow \mathrm{solve}(R^{\mathsf{T}}, A^{\mathsf{T}}C)$
      $Y_b \leftarrow Y_b + A\,\mathrm{solve}(R, FZ)$
    end if
  else
    $G \leftarrow A^{\mathsf{T}}A$; $H \leftarrow A^{\mathsf{T}}C$; $R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$
    $Z \leftarrow \mathrm{solve}(R^{\mathsf{T}}, H)$; $W \leftarrow \mathrm{solve}(R, (F - \eta\mathbf{I}_r)Z)$
    $Y_b \leftarrow \eta C + AW$
  end if
end for
return $\{Y_b\}_{b=1}^s$

For $N > r$, the backward below uses A, C, R, W, Z and the shared F. Implementations can save these values or reconstruct them from projections and compact intermediates. Equilibrium also reuses L and the LU factors of $\mathbf{I}_r + J$. No token-sized frame is needed. For $N \le r$, automatic differentiation handles the short-sequence expression, with an analytic adjoint for the shared core solve.

B.3 A common compact backward

Algorithm 3 differentiates Eq. (21) for one sample and head. It applies to both modes: set $F = 0$ for Complement and discard the adjoint of this fixed matrix. A bar denotes a loss adjoint under the Frobenius inner product, and $\mathrm{sym}(T) = (T + T^{\mathsf{T}})/2$. The algorithm includes both the direct content term and the correction. Their contributions to C and η are added before returning. Projection gradients follow the usual affine-layer rules.

For an upper Cholesky factor R, define

$$\Phi_u(T) = \mathrm{triu}(T, 1) + \tfrac{1}{2}\,\mathrm{diag}(\mathrm{diag}\,T), \qquad \mathcal{C}(R, E_R) = R^{-1}\,\mathrm{sym}\big(\Phi_u(E_R R^{\mathsf{T}})\big)\,R^{-\mathsf{T}}. \tag{39}$$

Here $\mathcal{C}$ is the adjoint of $R = \mathrm{chol}_u(S)$ with respect to the symmetric matrix S; the inverse notation again means two triangular solves. To obtain it, differentiate $S = R^{\mathsf{T}}R$ and observe that $dR = \Phi_u(R^{-\mathsf{T}}(dS)R^{-1})R$. This also fixes the half-diagonal convention in Eq. (39).

Algorithm 3 Rank-space backward ($N > r$, one sample and head)

Input: $E = \overline{Y} \in \mathbb{R}^{N\times h}$; forward values A, C, R, W, Z, F, η; fixed mask m and valid count n
Output: $\overline{B}, \overline{C}_{\mathrm{in}}$; contributions $\overline{\eta}, \overline{F}$

$E_W \leftarrow A^{\mathsf{T}}E$; $\overline{A} \leftarrow EW^{\mathsf{T}}$; $\overline{C} \leftarrow \eta E$
$\overline{\eta} \leftarrow \langle E, C\rangle_F$
$E_T \leftarrow \mathrm{solve}(R^{\mathsf{T}}, E_W)$
$\overline{F} \leftarrow E_T Z^{\mathsf{T}}$; $\overline{\eta} \leftarrow \overline{\eta} - \langle E_T, Z\rangle_F$
$\overline{Z} \leftarrow (F - \eta\mathbf{I}_r)^{\mathsf{T}}E_T$; $\overline{H} \leftarrow \mathrm{solve}(R, \overline{Z})$
$\overline{R} \leftarrow -E_T W^{\mathsf{T}} - Z\overline{H}^{\mathsf{T}}$
$\overline{G} \leftarrow \mathcal{C}(R, \overline{R})$ ▷ Eq. (39)
$\overline{A} \leftarrow \overline{A} + C\overline{H}^{\mathsf{T}} + A(\overline{G} + \overline{G}^{\mathsf{T}})$
$\overline{C} \leftarrow \overline{C} + A\overline{H}$
$\overline{B} \leftarrow m \odot \overline{A}/\sqrt{\max(n, 1)}$
$\overline{C}_{\mathrm{in}} \leftarrow m \odot \overline{C}$
return $\overline{B}, \overline{C}_{\mathrm{in}}, \overline{\eta}, \overline{F}$

Reusing the inverse triangular factor. The GPU path computes $T = R^{-\mathsf{T}}$ once and reuses it in forward and backward products. The compact derivatives above can be evaluated directly in terms of T. Let $M_F = F - \eta\mathbf{I}_r$, $E_T = TE_W$, and $V = E_T Z^{\mathsf{T}}$. Then

$$\overline{H} = T^{\mathsf{T}} M_F^{\mathsf{T}} E_T, \tag{40}$$

$$\overline{G} = T^{\mathsf{T}}\,\mathrm{sym}_\ell(-M_F V^{\mathsf{T}} - M_F^{\mathsf{T}} V)T, \tag{41}$$

where $\mathrm{sym}_\ell(K) = [\mathrm{tril}(K) + \mathrm{tril}(K, -1)^{\mathsf{T}}]/2$. The core and scalar contributions reuse V: $\overline{F} = V$ and $\overline{\eta} = \langle E, C\rangle_F - \mathrm{tr}(V)$. This form shares the same compact adjoint between Equilibrium and Complement.

B.4 Shared-core and scalar gradients

Write $M = \mathbf{I}_r + J$. For $F = M^{-1}(\mathbf{I} - J)$, differentiation gives

$$dF = -M^{-1}(dJ)(\mathbf{I} + F). \tag{42}$$

Since the core is shared, its adjoints are summed over samples before the transposed solve in Algorithm 4. The reduction is a sum: any loss averaging is already contained in $\overline{Y}$. The fast path accumulates this reduction in tiles rather than storing one $r \times r$ core adjoint per sample.

For either mode, sum the scalar adjoints over samples before applying $\overline{\eta_{\mathrm{raw}}} = (d\eta/d\eta_{\mathrm{raw}})\sum_b \overline{\eta}_b$. For the mathematical parameterization, $d\eta/d\eta_{\mathrm{raw}} = 1 - \tanh^2(\eta_{\mathrm{raw}})$. The fast path provides first-order training gradients.

Algorithm 4 Shared-core backward (once per head, Equilibrium only)

Input: $E_F = \sum_b \overline{F}_b$; Θ, L, F and the saved factorization of $M = \mathbf{I}_r + J$
Output: Core-parameter gradient $\overline{\Theta} \in \mathbb{R}^{r\times r}$
$T \leftarrow \mathrm{solve}(M^\mathsf{T}, E_F)$ ▷ Reuse the forward LU factors
$\overline{J} \leftarrow -T(\mathbf{I}_r + F)^\mathsf{T}$
$\overline{L} \leftarrow (\overline{J} + \overline{J}^\mathsf{T})L$
$d \leftarrow \mathrm{diag}(\overline{L}) \odot \mathrm{sigmoid}(\mathrm{diag}\,\Theta + \log(e-1))$
$\overline{\Theta} \leftarrow \mathrm{tril}(\overline{L}, -1) + \mathrm{diag}(d) + \mathrm{triu}(\overline{J} - \overline{J}^\mathsf{T}, 1)$
return $\overline{\Theta}$

B.5 Sequence-parallel reductions

Algorithm 5 expresses the operator for a sum of local losses. A global mean is obtained by normalizing every local loss with the same global denominator. VJP_W denotes the adjoint of $(G, H, F, \eta) \mapsto W$ in Eq. (21). It is the compact part of Algorithm 3 and excludes the direct term ηC. A sum reduction of E_W gives every device the same compact adjoints. The core adjoint is therefore global. The scalar adjoint also includes a sum of local direct-content contributions. Each shared-parameter gradient must be counted once. The implementation divides their replicated contributions by the number of devices before the final parameter-gradient sum.

Algorithm 5 Exact sequence-parallel MemSolve (one sample and head)

Input: Local rows B_s, C_s, mask m_s; replicated F, η; p devices; global physical length $N > r$
Output: Local output Y_s, local input gradients, and global parameter gradients
$n \leftarrow \mathrm{AllReduceSum}(\sum_i m_{s,i})$
$A_s \leftarrow (m_s \odot B_s)/\sqrt{\max(n,1)}$; $C_s \leftarrow m_s \odot C_s$
$(G, H) \leftarrow \mathrm{AllReduceSum}(A_s^\mathsf{T} A_s, A_s^\mathsf{T} C_s)$
$R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$; $Z \leftarrow \mathrm{solve}(R^\mathsf{T}, H)$
$W \leftarrow \mathrm{solve}(R, (F - \eta\mathbf{I}_r)Z)$; $Y_s \leftarrow \eta C_s + A_s W$
Backward: receive local output adjoint E_s
$E_W \leftarrow \mathrm{AllReduceSum}(A_s^\mathsf{T} E_s)$
$(\overline{G}, \overline{H}, \overline{F}, \overline{\eta}_W) \leftarrow \mathrm{VJP}_W(G, H, F, \eta; E_W)$
$\overline{A}_s \leftarrow E_s W^\mathsf{T} + C_s \overline{H}^\mathsf{T} + A_s(\overline{G} + \overline{G}^\mathsf{T})$
$\overline{C}_s \leftarrow \eta E_s + A_s \overline{H}$
$\overline{\eta} \leftarrow \mathrm{AllReduceSum}(\langle E_s, C_s\rangle_F) + \overline{\eta}_W$
$\overline{B}_s \leftarrow m_s \odot \overline{A}_s/\sqrt{\max(n,1)}$; $\overline{C}_{\mathrm{in},s} \leftarrow m_s \odot \overline{C}_s$
Backpropagate local projection gradients and sum them across devices
Apply the shared-core adjoint to $\overline{F}$ once; retain one copy of $\overline{\eta}$

The proof is the additivity of G, H in the forward pass and the chain rule in the backward pass. No recurrence connects neighboring sequence shards. In a backbone with a centered convolution, that local operation requires a separate halo exchange; it does not change the memory reduction. The implemented sequence-parallel path uses the compact formulation and requires global physical length greater than the operator rank.

B.6 Precision and scheduling

The fast path supports ranks 16, 32, 48, and 64. Token projections and statistics use BF16 multiplicands and FP32 accumulation. Gram matrices, compact states, factors, and adjoints retain FP32 storage. Batched compact products use Triton's three-term TF32 decomposition; small shared-core products and solves retain IEEE FP32 arithmetic. Readout kernels split sensitive FP32 coefficients into BF16 leading and residual words before accumulating both products in FP32. The extra product recovers coefficient precision without using FP32 operands for token-sized products. With very sparse masks and strong relations, cancellation can still amplify relative error in the compact complementary readout.

Training implementation. The completed training runs used NVIDIA A800-SXM4-80GB GPUs, PyTorch 2.14.0 with CUDA 13.2, and timm 1.0.29. Compilation fuses surrounding operations; CUDA Graph replay reduces launch overhead. These execution choices do not alter Eqs. (5)–(13).

B.7 Optional rotary value encoding

Let $\mathcal{O}$ rotate each content row: $(\mathcal{O}C)_i^{\mathsf{T}} = O_i c_i$. The rotations cover all h channels in adjacent pairs, so h is even. The compact forward changes only the content statistic and final readout:

$$\begin{aligned} \widetilde{C} &= \mathcal{O}C, & H &= A^{\mathsf{T}}\widetilde{C}, & Z &= R^{-\mathsf{T}}H, \\ W &= R^{-1}(F - \eta \mathbf{I}_r)Z, & Y &= \eta C + \mathcal{O}^{-1}(AW). \end{aligned} \tag{43}$$

The Gram matrix and core solve are unchanged. Both modes use this form, with $F = 0$ for Complement. The inverse rotation covers the full correction, including its $-\eta\mathbf{I}_r$ term, and precedes the output projection. Backward applies the transposed rotations around the same compact adjoints. The rotations cost $O(Nh)$ and fuse into statistics and readout. They are evaluated in FP32. Products involving rotated content use BF16 leading and residual words, as in Appendix B.6. Sequence shards use global positions and the same frequency scale.

For a fixed frame, write $\mathcal{K}(C) = K_X C$ with K_X from Eq. (15). Equation (43) is $\mathcal{O}^{-1}\mathcal{K}\mathcal{O}$. Since $\mathcal{O}$ is orthogonal under the Frobenius inner product, this conjugation preserves singular values and the gain bound in Theorem 3.2.

We follow RoVE's value rotations and upstream frequency interpolation [10, 27]. Frequencies are fixed at $\omega_k = 10000^{-2k/h}$ for $k = 0, \ldots, h/2 - 1$. The optional language configuration uses a 128-token training window. Beyond it, frequency-only YaRN retains high frequencies and smoothly scales low frequencies, with $\beta_{\text{fast}} = 32$ and $\beta_{\text{slow}} = 1$. Forward and inverse rotations use the same frequency table and unit amplitude. This implemented extension has not yet been evaluated in the full pretraining and downstream experiments reported here.

C Experimental Protocols and Results

C.1 Visual models and optimization

Table 4 summarizes the architecture and ImageNet-1K training recipe for all three model sizes. The completed Small and Base runs provide the results in Table 1.

Configuration	MemSolve-ViT-T	MemSolve-ViT-S	MemSolve-ViT-B
Width	192	384	768
Heads	6	6	12
Rank per head	16	32	48
FFN width	768	1,584	3,072
Batch size		2,048	
Optimizer		AdamW	
Loss		Cross-entropy	
Learning rate		2×10^{-3}	
Learning-rate decay		Cosine	
Weight decay		0.05	
Warmup epochs		20	
Input size		224×224	
Epochs		300	
Label smoothing		0.1	
Stochastic depth	0.0	0.1	0.4
Repeated augmentation		✓	
Gradient clipping		5.0	
EMA decay		0.9999	
Horizontal flip		✓	
Random resized crop		✓	
RandAugment		✓	
Mixup α [45]		0.8	
CutMix α [44]		1.0	
Random erasing probability [46]		0.25	
Test crop ratio		0.875	

Table 4: **MemSolve-ViT configurations and ImageNet-1K training recipe.** All models use 12 blocks and 16×16 patches. Shared settings span all three columns. Stochastic depth increases linearly to the listed maximum.

Repeated augmentation [18] uses three repeats with a fixed epoch budget. RandAugment [5] uses `rand-m9-mstd0.5-inc1`, without separate color jitter. Warmup replaces the first 20 epochs of the 300-epoch cosine schedule. EMA uses its fixed decay from the first optimizer step. ImageNet training uses seed 0.

Baseline reporting. Table 1 contains non-hierarchical models only. "iso." denotes isotropic ConvNeXt. The Type column uses SSM for state-space models, Linear for linear-attention and linear-recurrence mixers, and TTT for test-time training. ViT-B and Hydra-ViT-B use the EMA entries in Hydra's Table 5 [20]. Vim's † entries include 30 additional epochs of long-sequence fine-tuning at 224×224, with patch size 16 and extraction stride 8 [47]. LION's ♮ entries use multiple scans [1]. Parameter counts are rounded to whole millions and Top-1 to one decimal place. Training protocols differ across papers.

Convergence logs. Figure 3 uses the DeiT authors' four-GPU Small log. The authors state that this run differs from the released checkpoint. The log contains 300 epochs and ordinary-weight validation. Zero-based DeiT epochs are shifted by one to denote completed epochs. MemSolve-S uses the ordinary-weight validation recorded during the same run as its EMA result in Table 1. Every recorded epoch is plotted without smoothing; missing epochs remain gaps.

C.2 Language pretraining

Table 5 lists the completed pretraining configuration. The encoder retains BERT's embedding, post-LayerNorm, and tied MLM-head conventions. Its residual short convolutions start as the identity; padding is masked before and after each convolution.

Configuration	MemSolve-BERT
Parameters (M)	112
Depth	14
Width	768
Heads	12
Rank per head	32
FFN	SwiGLU, width 2,048 per branch
Position embedding	Learned absolute, 512 positions
Short convolution	Centered depthwise, kernel size 3
Pretraining corpus	C4 (English)
Sequence length	128
Batch size	4,096
Training steps	70,000
Training mask probability	0.30
Validation mask probability	0.15
Optimizer	AdamW
AdamW (β_1, β_2)	$(0.9, 0.98)$
Learning rate	8×10^{-4}
Learning-rate decay	Linear
Final learning rate	1.6×10^{-5}
Warmup steps	4,200 (6%)
Weight decay	0.0125
Dropout	0.1
Gradient clipping	1.0

Table 5: **MemSolve-BERT masked-language pretraining recipe.** Parameters include the tied MLM head and are rounded to whole millions.

Documents are streamed and truncated individually, without packing. MLM uses the standard 80/10/10 mask/random/unchanged corruption rule. Validation uses fixed corruption on 10,000 documents, excludes padding and special tokens from targets, and averages loss over masked targets. The checkpoint with the lowest validation masked NLL is step 70,000.

Weight decay converts LION's Composer value as $10^{-5}/(8 \times 10^{-4}) = 0.0125$; biases and normalization parameters are excluded. The final learning rate follows the public code's 2% endpoint rather than the 20% stated in the LION paper.

C.3 GLUE fine-tuning and repeated seeds

The BERT and LION baselines in Table 2 are taken from LION's Table 10, M2-BASE rows [1]. MemSolve reports per-task means; Table 7 lists the metrics, seed sets, and standard deviations.

The classifier uses BERT's CLS pooler (dense plus $\tanh$), dropout, and a task-specific linear head. All encoder weights transfer from MLM; the absent pooler and classifier are initialized with BERT's initializer. For MNLI transfer, the trained pooler is retained and the classifier is reset. STS-B uses mean squared error; other tasks use cross-entropy.

Table 6: **GLUE fine-tuning configuration.** WD lists the source recipe value. For Composer rows, PyTorch WD equals the listed value divided by peak LR; STS-B uses its listed PyTorch value directly.

Task	Start	Epochs	Batch	Peak LR	WD	WD convention
MNLI	MLM	3	48	$5 \cdot 10^{-5}$	$5 \cdot 10^{-6}$	Composer
QNLI	MLM	10	16	$5 \cdot 10^{-5}$	10^{-5}	Composer
QQP	MLM	5	16	$3 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	Composer
RTE	MNLI	3	16	10^{-5}	10^{-6}	Composer
SST-2	MLM	3	16	$3 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	Composer
MRPC	MNLI	10	32	$8 \cdot 10^{-5}$	$8 \cdot 10^{-5}$	Composer
CoLA	MLM	10	32	$8 \cdot 10^{-5}$	$5 \cdot 10^{-6}$	Composer
STS-B	MNLI	10	32	$8 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	PyTorch

All tasks use sequence length 128, AdamW betas $(0.9, 0.98)$, $\epsilon = 10^{-6}$, 6% step warmup, linear decay to zero, and no gradient clipping. Weight decay applies to all parameters in fine-tuning, as in the source GLUE jobs. Sequential sampling keeps partial final batches. Evaluation uses the final checkpoint on the development set; no hidden test labels are used. Fine-tuning uses eager execution of the BERT shell with the native CUDA mixer and BF16 autocast.

Table 7: **MemSolve GLUE seed aggregation.** SD is the sample standard deviation over fine-tuning seeds. A dash denotes one run. Seed sets are $\mathcal{S}_1 = \{19\}$, $\mathcal{S}_3 = \{19, 8364, 717\}$, $\mathcal{S}_4 = \mathcal{S}_3 \cup \{10536\}$, and $\mathcal{S}_5 = \mathcal{S}_4 \cup \{90166\}$.

Task	Metric	Seeds	Mean	SD
MNLI	Matched/mismatched accuracy mean	$\mathcal{S}_1$	83.0	—
QQP	Accuracy/F1 mean	$\mathcal{S}_1$	89.8	—
QNLI	Accuracy	$\mathcal{S}_1$	88.6	—
SST-2	Accuracy	$\mathcal{S}_3$	91.6	0.350
CoLA	Matthews correlation	$\mathcal{S}_4$	47.5	1.505
RTE	Accuracy	$\mathcal{S}_5$	74.2	0.791
MRPC	F1	$\mathcal{S}_5$	91.0	0.653
STS-B	Spearman correlation	$\mathcal{S}_5$	89.2	0.140

All runs share the seed-17 pretrained checkpoint. The MNLI-transfer tasks also share the seed-19 MNLI parent checkpoint. Their SD therefore measures variation in the final task's fine-tuning, not independent pretraining or independent MNLI transfer. The 81.9 aggregate averages the eight task means and has no suite-level SD because task seed counts differ.

C.4 Efficiency benchmark

Mixers and shapes. We benchmark complete mixers with input and output projections. MemSolve uses Equilibrium without RoVE or an external positional module. FlashAttention uses `timm` attention with packed QKV projections and PyTorch's FlashAttention-2 SDPA backend, without a causal mask. BiMamba uses Vim's fused v2 implementation: expansion factor two, state size 16, convolution width four, and averaged directional outputs. It shares input and output projections across directions. The three mixers receive identical input shapes. The 196-token visual settings use $(B, D, H, r) = (128, 192, 6, 16)$, $(128, 384, 6, 32)$, and $(64, 768, 12, 48)$ for Tiny, Small, and Base. Here B is batch size, D is width, H is the head count for MemSolve and attention, and r is MemSolve's rank. The length sweep fixes $(B, D, H, r) = (4, 768, 12, 32)$ and varies length from 128 to 32,768.

Execution and measurement. All runs use one NVIDIA A800-SXM4-80GB, PyTorch 2.14.0 with CUDA 13.2, FP32 parameters, BF16 inputs and autocast, and enabled TF32. All three mixers use CUDA Graphs without `torch.compile`. Forward–backward timing includes the mean squared output as a scalar loss and its backward pass,

Table 8: **Forward-only mixer latency and peak memory.** Settings match Table 3.

	MemSolve		FlashAttention		BiMamba	
Setting	ms	MiB	ms	MiB	ms	MiB
Visual shapes: 196 tokens						
Tiny	**0.169**	**66.4**	0.208	80.6	2.143	209.7
Small	0.418	**109.2**	**0.343**	138.0	4.244	394.9
Base	0.724	**131.3**	**0.477**	146.1	4.433	405.0
Length sweep: batch 4, width 768						
128 tokens	0.165	**35.1**	**0.054**	39.8	0.269	55.3
196 tokens	0.167	**36.9**	**0.074**	41.4	0.446	63.4
512 tokens	0.200	**45.2**	**0.134**	52.9	0.548	98.4
2,048 tokens	**0.334**	**86.0**	0.589	106.5	1.632	270.7
8,192 tokens	**0.768**	**247.2**	5.346	322.5	6.179	962.2
32,768 tokens	**2.357**	**895.2**	75.901	1186.5	25.735	3728.7

without an optimizer step. Forward-only timing uses inference mode. Each process performs 20 warmup steps, captures the computation, then performs 20 graph warmups. CUDA events measure seven groups of 50 replays. We report the mean of the median latencies from two independent processes, reversing operator order in the second repetition. For example, the two MemSolve forward–backward measurements at 2,048 tokens are 0.918 and 1.076 ms; their mean is 0.997 ms. We reset PyTorch's peak allocated memory after the initial warmup and measure through graph capture and replay. This includes parameters, gradients, and intermediates; allocator reserves and the CUDA context are excluded. Tables 3 and 8 report forward–backward and forward-only measurements, respectively.

Implementation provenance. BiMamba comes from Vim commit `dd0358a`, with the upstream fused selective-scan and causal-convolution kernels. We adapt its convolution API calls to the current upstream allocation interface. Forward and gradient comparisons check this adaptation, and profiling confirms native scan and convolution kernels. Profiling also confirms the FlashAttention backend. The benchmark records retain source snapshots, compatibility patches, commands, and both timing repetitions.

C.5 Reproducibility

The training records preserve resolved recipes, source snapshots, data manifests, and checkpoint selection. The accompanying repository provides model code, training entrypoints, and benchmark code at `https://github.com/Yang916-yy/MemSolve`. Figures are vector schematics of the equations and plots of all recorded epochs without smoothing. Scientific Agent Skills [21] provided writing and figure-review procedures for this revision.

D Extended Related Work

D.1 Internal optimization and equilibrium models

Internal learning can be implemented by solving for a state and then reading from it. Intention makes this construction explicit in the key–value–query setting [11]. It fits a regularized regression model to embedded key–value pairs and evaluates the fitted model at query embeddings. A learned transformation can further process the fitted state. The solve thus becomes a composable network module, connecting internal learning to attention-like computation.

Equilibrium models study how such internal states are defined and computed. Deep equilibrium models define hidden states through fixed points [2], while monotone equilibrium models use operator structure to guarantee well-posedness [39]. These approaches make the properties of the state equation part of the model design.

MemSolve connects these perspectives by designing the memory equation together with its token readout. The same input-dependent frame writes associations into memory and reads the response back to tokens. Matching the regularizer to this frame makes the system matrix shared across inputs, while the associations determine its right-hand side. The accretive structure gives a unique state. The reflected readout then links that structure to a bound on the output gain through the classical accretive–Cayley correspondence [3]. This joint design connects the solvability of the internal problem to the behavior of the resulting token map.

D.2 Kernel-based operator learning

Normalized feature coordinates provide another connection: they can gather values into a compact representation and redistribute them across positions. ONO uses this structure to learn solution operators for partial differential equations [41]. It represents a kernel through learned eigenfunctions and positive eigenvalue weights. Cholesky whitening uses covariance statistics aggregated over training samples and positions; running statistics are reused at inference. Positive weights give a symmetric positive-semidefinite kernel on the shared feature coordinates.

MemSolve instead normalizes its frame using the relation statistics of each input. The identity shift produces a soft normalization: singular values remain below one rather than being set to one. The resulting frame therefore adapts the compact representation to the current context. Within that representation, the Cayley-parameterized core can express any real strict contraction, including nonsymmetric maps. The connection to ONO lies in gathering and redistributing content through normalized features. The normalization statistics and the map between these features determine different forms of token interaction.

D.3 Bidirectional state-space and linear-recurrence models

Structured SSMs provide efficient sequence maps [14, 13, 7]. Vim adapts selective scans to vision, Hydra develops bidirectional matrix mixers, and LION relates full-attention, recurrent, and chunkwise forms [47, 20, 1]. Video DeltaNet combines local attention with bidirectional memory updated jointly within each frame [40].

For token interactions, the key question is how the two directional responses are combined. With local parameterization, independent scans, and linear merging, a fixed center token receives an additive prefix response and suffix response. Changing the suffix leaves the prefix response unchanged.

MemSolve uses the full input to normalize its frame. A suffix change can therefore alter how prefix content contributes to the same center token. Theorem 3.3 establishes this distinction for the single-layer scan class above. It is a distinction in the dependence of the output on the input, rather than a choice between recurrent and parallel evaluation. An equivalent recurrent or chunkwise implementation preserves the interactions of its underlying operator. MemSolve's compact statistics allow these context-dependent interactions to be evaluated in parallel, with linear cost in sequence length at fixed rank.